

BỘ GIÁO DỤC VÀ ĐÀO TẠO
ĐẠI HỌC PHENIKAA



**BÁO CÁO GIỮA HỌC PHẦN: ỨNG
DỤNG PHÂN TÁN**
**TÊN ĐỀ TÀI: XÂY DỰNG HỆ THỐNG LƯU TRỮ
KEY-VALUE PHÂN TÁN**

Nhóm sinh viên thực hiện

Sinh viên	Mã sinh viên
Nguyễn Tuấn Thiên	23010571
Đỗ Trung Kiên	23010516

Khóa: 17

Hệ: Đại học chính quy

Giảng viên hướng dẫn: TS. Nguyễn Lệ Thu

Hà Nội – 2026

BẢNG PHÂN CÔNG NHIỆM VỤ

Thành viên	Nội dung đóng góp chính
Nguyễn Tuấn Thiên - 23010571	Khởi tạo dự án & dependencies; scaffold node server và cấu trúc KeyValueNode; cài đặt luồng PUT (forward + helper nhân bản), GET (local-first + fallback Replica), DELETE (lan Primary-Replica); RPC snapshot dữ liệu & thông tin định tuyến; vòng lặp Heartbeat & phát hiện lỗi; recovery sync + force-sync RPC; entypoint node & parse cấu hình; client (nạp config, phân giải endpoint, CLI tương tác); Manager (API trạng thái cụm, API CRUD forwarding) và toàn bộ giao diện Web (templates, CSS, script frontend); bulk routing API + nhân bản bù tự phục hồi (tính năng mới 12.1); test cụm & healthcheck; cấu hình Docker, scripts và tài liệu triển khai.
Đỗ Trung Kiên - 23010516	Khởi tạo repository (Initial commit, đã verified); bổ sung các hàm helper định tuyến & RPC trong KeyValueNode; lưu trữ bền vững (persistence) bằng cách ghi/đọc snapshot JSON ra đĩa; điều khiển container Docker từ Manager API & UI phục vụ thêm/xóa node động (góp phần cho tính năng mới 12.2); cập nhật volume Docker & scripts điều khiển container; tài liệu hướng dẫn cài đặt Docker & changelog; cấu hình Docker, scripts và tài liệu triển khai.

MỤC LỤC

TÊN ĐỀ TÀI: XÂY DỰNG HỆ THỐNG LƯU TRỮ KEY-VALUE PHÂN TÁN.....	1
Nhóm sinh viên thực hiện.....	1
BẢNG PHÂN CÔNG NHIỆM VỤ.....	2
TỔNG QUAN.....	4
1. Giới thiệu.....	6
2. Kiến trúc tổng thể hệ thống.....	7
2.1. Mô hình kiến trúc phân tán (Peer-to-Peer).....	7
2.2. Thành phần chi tiết của mỗi Node.....	7
2.3. Giao thức truyền thông XML-RPC.....	7
2.4. Cấu hình và khởi tạo cụm.....	7
3. Mô hình lưu trữ, phân mảnh và sao lưu.....	8
3.1. Consistent Hashing và Virtual Nodes.....	8
3.2. Mô hình sao lưu (Replication).....	9
3.3. Mô hình nhất quán cuối cùng (Eventual Consistency).....	9
4. Định tuyến và chuyển tiếp yêu cầu.....	9
4.1. Định tuyến PUT và GET.....	9
4.2. Dự phòng lỗi khi định tuyến (Failover Routing).....	11
5. Phát hiện lỗi và xử lý node chết.....	11
5.1. Heartbeat và phát hiện lỗi tự động.....	11
5.2. Kịch bản khi node chết và cơ chế nhân bản bù.....	11
6. Khôi phục và đồng bộ lại dữ liệu.....	12
6.1. Đồng bộ khi khởi động và dọn bản thừa.....	12
6.2. Chống ghi đè dữ liệu cũ khi hồi phục.....	13
7. Quản lý cụm động: Thêm / Xóa / Khôi phục node.....	13
8. Phân tích theo định lý CAP.....	15
9. Ứng dụng quản trị trực quan (Web Dashboard).....	15
9.1. Tối ưu hiệu năng tải.....	16
10. Cài đặt, triển khai và kết quả.....	17
10.1. Yêu cầu môi trường và cách cài đặt.....	17
10.2. Kịch bản kiểm thử.....	17
10.3. Kết quả đạt được.....	17
11. Đánh giá hạn chế và đề xuất cải tiến.....	18
12. Tính năng mới phát triển thêm.....	18
12.1. Tự phục hồi dữ liệu: nhân bản bù tự động và dọn bản thừa (self-healing).....	18
12.2. Quản lý cụm động (dynamic membership): thêm / xóa / khôi phục node khi đang chạy.....	18
Kết luận.....	20

TỔNG QUAN

What - Dự án là gì. ARS Store là một kho lưu trữ key-value phân tán: thay vì một máy chủ duy, dữ liệu được trải trên nhiều node ngang hàng (peer-to-peer) hợp thành một cụm. Mỗi cặp key-value được một node làm chủ (Primary) và được sao lưu sang một node khác (Replica) để an toàn. Hệ thống cung cấp ba thao tác cơ bản PUT / GET / DELETE, tự định tuyến yêu cầu tới đúng node, tự phát hiện node chết và phục vụ tiếp từ bản sao, tự nhân bản bù để luôn đủ số bản, cho phép thêm/xóa/khôi phục node ngay khi đang chạy, và đi kèm một giao diện web (Flask) để giám sát và vận hành cụm.

Why - Vì sao làm. Mục đích là khắc phục nhược điểm của lưu trữ tập trung: điểm lỗi duy nhất (một máy sập là sập cả dịch vụ), khó mở rộng, và nghẽn cổ chai khi tải cao. Lý do cho từng lựa chọn kỹ thuật: dùng consistent hashing vì khi thêm/bớt một node chỉ khoảng k/n key phải dời chỗ, trong khi cách ngẫu nhiên $\text{hash} \% N$ phải băm lại gần như toàn bộ dữ liệu; dùng virtual node (256/node) để phân bố tải đều, tránh node này gánh quá nhiều còn node kia quá ít; dùng nhân bản (`replication_factor=2`) để dữ liệu không mất khi một node chết; và chọn hướng ưu tiên tính sẵn sàng (AP trong định lý CAP, nhất quán cuối cùng) để hệ thống vẫn đọc/ghi được ngay cả lúc có node hỏng.

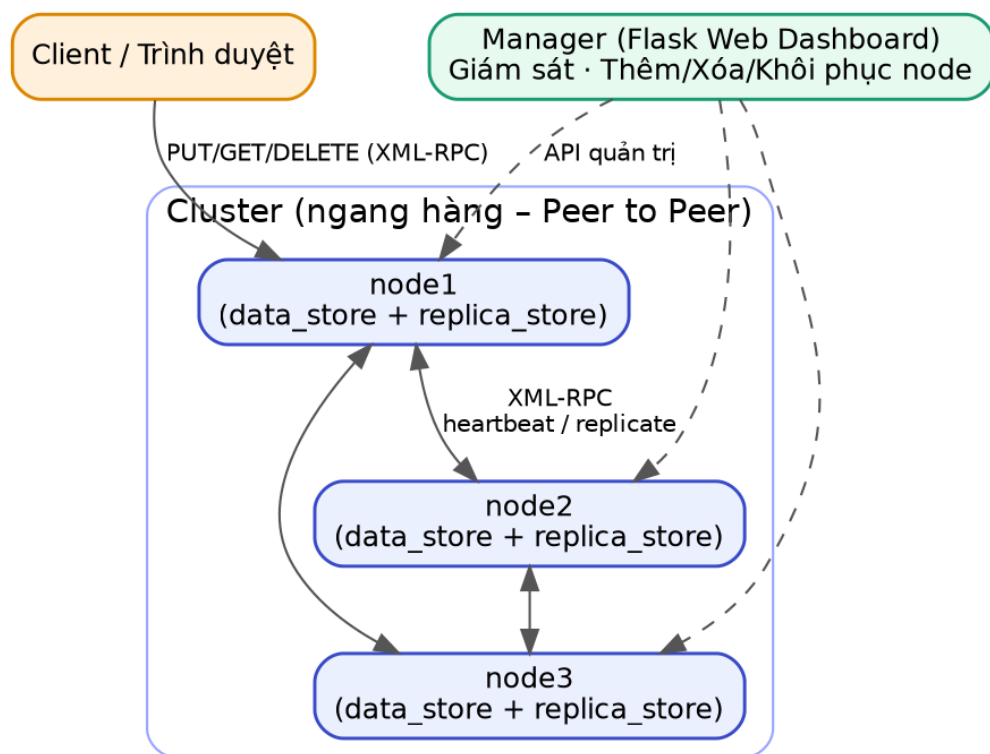
When — Khi nào dùng. Khi client gửi PUT/GET: node tiếp nhận tính node chủ rồi xử lý hoặc chuyển tiếp. Khi một node ngừng trả lời quá `failure_timeout` (5 giây): heartbeat đánh dấu DEAD, các key của nó được phục vụ từ bản sao, và cứ mỗi ~2 giây hệ thống nhân bản bù để mỗi key đủ 2 bản sống. Khi cụm khỏe trở lại (không còn node chết): cơ chế dọn bản thừa gom dữ liệu về đúng số bản. Khi thêm/xóa/khôi phục node: vòng băm được dựng lại và chỉ phần dữ liệu cần thiết được di trú. Khi một node hồi phục: nó đồng bộ lại và ưu tiên giá trị từ các node đã sống (vì lúc nó chết, ghi mới chỉ tới được node sống).

Về phạm vi áp dụng: hệ thống phù hợp cho các kho key-value cần sẵn sàng cao, chịu được node hỏng, và có thể mở rộng quy mô bằng cách thêm node.

How — Hoạt động thế nào. Hệ thống băm cả key lẫn ID node bằng MD5 lên một vòng băm, mỗi node vật lý rải thành 256 virtual node. Để tìm Primary của

một key, bấm key rồi đi theo chiều kim đồng hồ tới virtual node gần nhất; Replica là node sống kế tiếp trên vòng. Các node giao tiếp với nhau và với client qua XML-RPC (có timeout để không bị treo khi gọi node đã sập), và dùng threading.Lock để tránh xung đột khi xử lý đồng thời.

Các luồng chính: PUT ghi vào Primary rồi nhân bản nền sang Replica (nếu node nhận không phải Primary thì forward; nếu Primary chết thì ghi suy giảm và lan sang tất cả node sống phụ trách key). Phát hiện lỗi bằng hai vòng lặp heartbeat (gửi nhịp tim + giám sát timeout). Khả năng tự chữa lành nhờ ba cơ chế chạy định kỳ: nhân bản bù khi thiếu, dọn bản thừa khi dư, và đồng bộ ưu tiên dữ liệu từ node sống khi hồi phục. Quản lý cụm động thực hiện bằng cách phát thông tin thành viên mới cho mọi node rồi di trú key về đúng chủ. Toàn bộ được giám sát qua một Manager Flask với các trang Dashboard, Danh sách dữ liệu, Trạng thái Server và Nhật ký hoạt động.



Hình 0. Kiến trúc tổng thể hệ thống ARS Store

1. Giới thiệu

1.1. Bài toán đặt ra

Trong các hệ thống lưu trữ tập trung truyền thống, toàn bộ dữ liệu được ghi và truy vấn qua một máy chủ duy nhất. Mô hình này bộc lộ nhiều nhược điểm khi mở rộng quy mô: điểm lỗi duy nhất (Single Point of Failure) khiến toàn bộ dịch vụ ngừng khi máy chủ sập; chi phí mở rộng theo chiều dọc đắt đỏ và bị giới hạn vật lý; và hiện tượng nghẽn cổ chai khi lưu lượng tăng cao.

Để khắc phục, nhóm xây dựng “Hệ thống lưu trữ Key-Value phân tán”: nhiều máy chủ độc lập (Node) hợp thành một cụm (Cluster), cùng chia sẻ tải, tự sao lưu dữ liệu và duy trì hoạt động liên tục ngay cả khi một số node hỏng.

1.2. Mục tiêu của hệ thống

- Triển khai cụm tối thiểu 3 node độc lập, ngang hàng, chia sẻ việc lưu trữ.
- Hỗ trợ đầy đủ PUT(key,value), GET(key), DELETE(key).
- Phân mảnh dữ liệu bằng Consistent Hashing với virtual node để cân bằng tải.
- Nhân bản mỗi bản ghi (replication_factor = 2) để đảm bảo an toàn dữ liệu.
- Phát hiện sự cố bằng Heartbeat và tự động chuyển phục vụ sang bản sao (failover).
- Tự nhân bản bù khi node chết và tự dọn bản thừa khi cụm khỏe để luôn đủ số bản sao.
- Thêm / Xóa / Khôi phục node ngay khi hệ thống đang chạy (dynamic membership).
- Cung cấp Web Dashboard trực quan để giám sát và vận hành cụm.

1.3. Ứng dụng trong thực tế

Kho key-value phân tán như hệ thống này là nền tảng của nhiều dịch vụ thực tế: bộ nhớ đệm phân tán (distributed cache) và lưu phiên đăng nhập (session store) cho web quy mô lớn; giỏ hàng, hồ sơ người dùng trong thương mại điện tử (mô hình từng được Amazon Dynamo áp dụng); lưu cấu hình/metadata dịch vụ, hàng đợi nhẹ, bảng xếp hạng (leaderboard) thời gian thực, hay dữ liệu đo từ thiết bị IoT. Các kỹ thuật mà đề tài cài đặt — consistent hashing để phân mảnh, nhân bản nhiều bản, phát hiện lỗi bằng heartbeat và tự phục vụ từ bản sao — chính là những kỹ thuật mà các hệ cơ sở dữ liệu phân tán thương mại như Amazon

DynamoDB, Apache Cassandra, Riak hay Redis Cluster dùng để đạt khả năng mở rộng ngang và sẵn sàng cao. Vì vậy hệ thống không chỉ có giá trị học thuật mà còn phản ánh sát kiến trúc của các kho dữ liệu phân tán đang vận hành trong thực tế.

2. Kiến trúc tổng thể hệ thống

2.1. Mô hình kiến trúc phân tán (Peer-to-Peer)

Hệ thống theo kiến trúc phi tập trung: không có node master điều phối trung tâm, mọi node ngang hàng nhau, vừa lưu trữ vừa tiếp nhận yêu cầu trực tiếp từ client. Client có thể gửi yêu cầu tới bất kỳ node nào; node tiếp nhận tự tính định tuyến và chuyển tiếp tới đúng node lưu trữ. Một thành phần Manager (ứng dụng Flask) đứng ngoài cụm chỉ làm nhiệm vụ giám sát và điều khiển vận hành (thêm/xóa/khôi phục node), không nằm trên đường dữ liệu chính.

2.2. Thành phần chi tiết của mỗi Node

Mỗi node là một đối tượng KeyValueNode gồm các thành phần cốt lõi:

- `data_store` (Primary Store): bộ nhớ chính chứa các key mà node trực tiếp phụ trách.
- `replica_store` (Replica Store): chứa bản sao của các key thuộc node khác.
- `neighbor_ids`: danh sách node lân cận để gửi heartbeat và đồng bộ.
- Hash ring: vòng băm với 256 virtual node cho mỗi node vật lý, dùng để định tuyến.
- Cơ chế Heartbeat: vòng lặp gửi nhịp tim định kỳ và vòng lặp giám sát timeout để cập nhật trạng thái ALIVE/DEAD.
- Lock đồng bộ (`threading.Lock`): bảo vệ bộ nhớ khỏi race condition khi xử lý đồng thời nhiều truy vấn.

2.3. Giao thức truyền thông XML-RP

Mọi giao tiếp Client–Node và Node–Node dùng XML-RPC trên nền HTTP/TCP, tận dụng thư viện chuẩn của Python (`xmlrpc.client`, `SimpleXMLRPCServer`). Một lớp `TimeoutTransport` tùy biến áp đặt giới hạn thời gian chờ kết nối, ngăn node bị treo vĩnh viễn khi gọi tới node đã sập.

2.4. Cấu hình và khởi tạo cụm

Toàn bộ cấu hình (danh sách node, host, port, `replication_factor`, các tham số heartbeat) khai báo tập trung trong `cluster_config.json`. Mỗi node khi khởi chạy

chỉ cần chỉ định ID và đường dẫn cấu hình:

```
python node.py --node <NODE_ID> --config cluster_config.json --bind-host <IP>
```

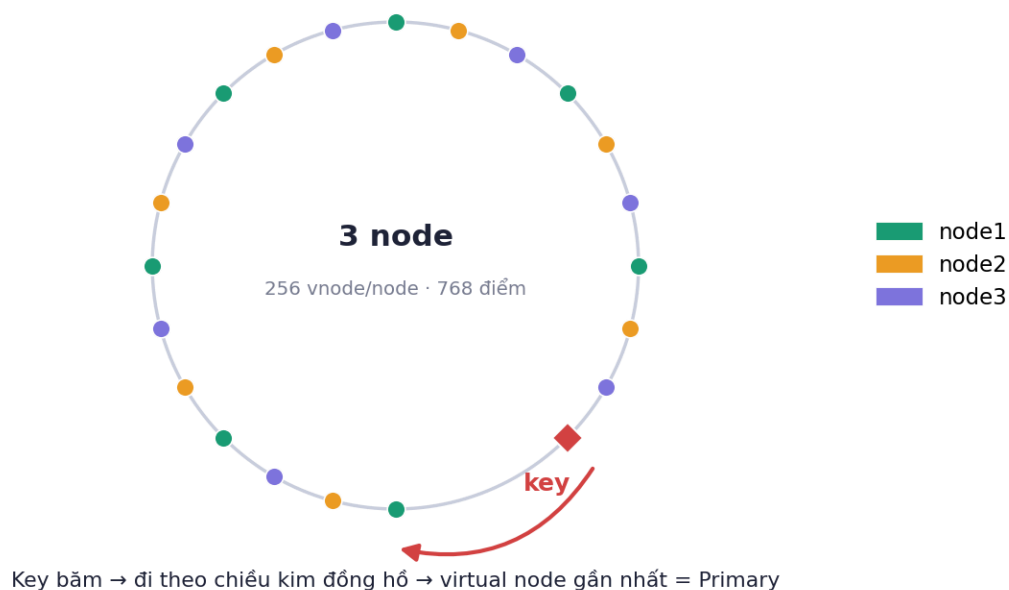
Cụm mặc định gồm node1/node2/node3. Tham số quan trọng: replication_factor = 2 (1 primary + 1 replica), heartbeat_interval = 2 giây, failure_timeout = 5 giây (sau khoảng này không nhận được nhịp tim thì node bị coi là DEAD).

3. Mô hình lưu trữ, phân mảnh và sao lưu

3.1. Consistent Hashing và Virtual Nodes

Hệ thống dùng hàm băm MD5 ánh xạ cả KEY lẫn ID node lên cùng một không gian băm tạo thành vòng tròn (hash ring). Thay vì đặt mỗi node tại một điểm duy nhất, mỗi node vật lý được rải thành 256 virtual node quanh vòng (cụm 3 node có $3 \times 256 = 768$ điểm). Việc này giúp phân bổ tải đều hơn nhiều so với một điểm/node.

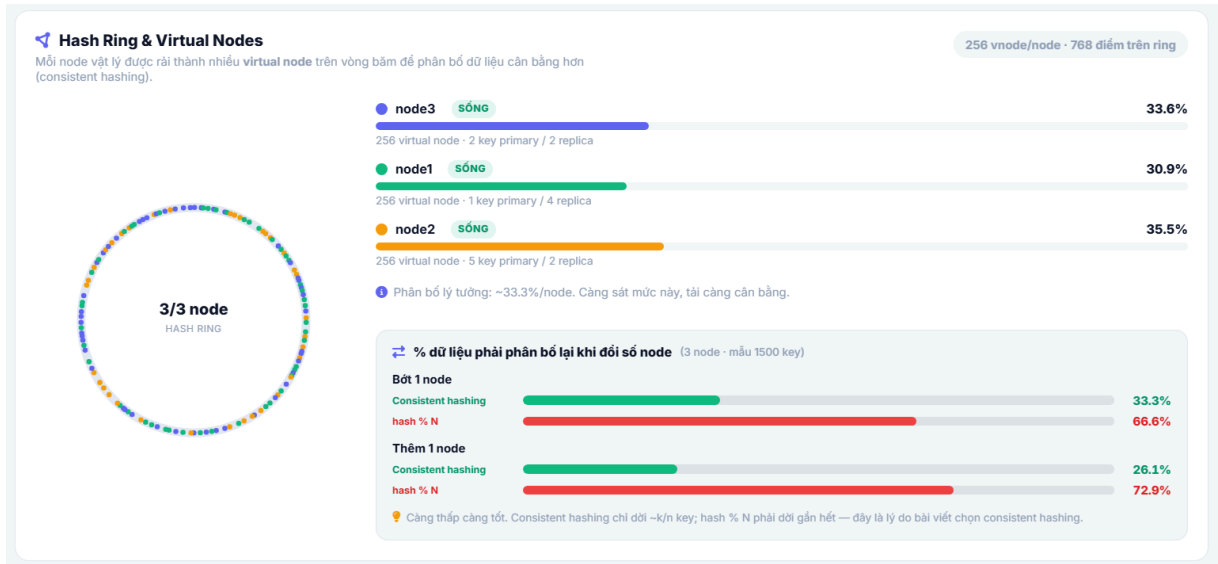
Để tìm Primary của một key: băm key ra một điểm trên vòng, rồi đi theo chiều kim đồng hồ tới virtual node gần nhất; node sở hữu virtual node đó là Primary. Cách này (khác hẳn phép chia dư % N) bảo đảm khi thêm/bớt một node thì chỉ khoảng k/n key phải di chuyển, thay vì phải băm lại gần như toàn bộ dữ liệu.



Hình 3.1. Vòng băm với virtual node — key đi theo chiều kim đồng hồ tới

Primary

Giao diện giám sát trực quan hóa vòng băm, tỉ lệ keyspace mỗi node (lý tưởng ~33,3%/node) và so sánh phần trăm dữ liệu phải di chuyển khi đổi số node giữa Consistent Hashing và hash % N — minh chứng vì sao consistent hashing tối ưu hơn:



Hình 3.2. Màn hình Hash Ring & Virtual Nodes trên Web Dashboard

3.2. Mô hình sao lưu (Replication)

Với replication_factor = 2, mỗi key được giữ trên đúng 2 node: 1 Primary và 1 Replica. Replica là node KẾ TIẾP theo chiều kim đồng hồ ngay sau Primary trên vòng băm. Khi Primary nhận PUT, nó ghi vào data_store rồi nhân bản (luồng nền) sang Replica; nếu Replica đang DOWN thì bỏ qua để không làm nghẽn phản hồi cho client.

3.3. Mô hình nhất quán cuối cùng (Eventual Consistency)

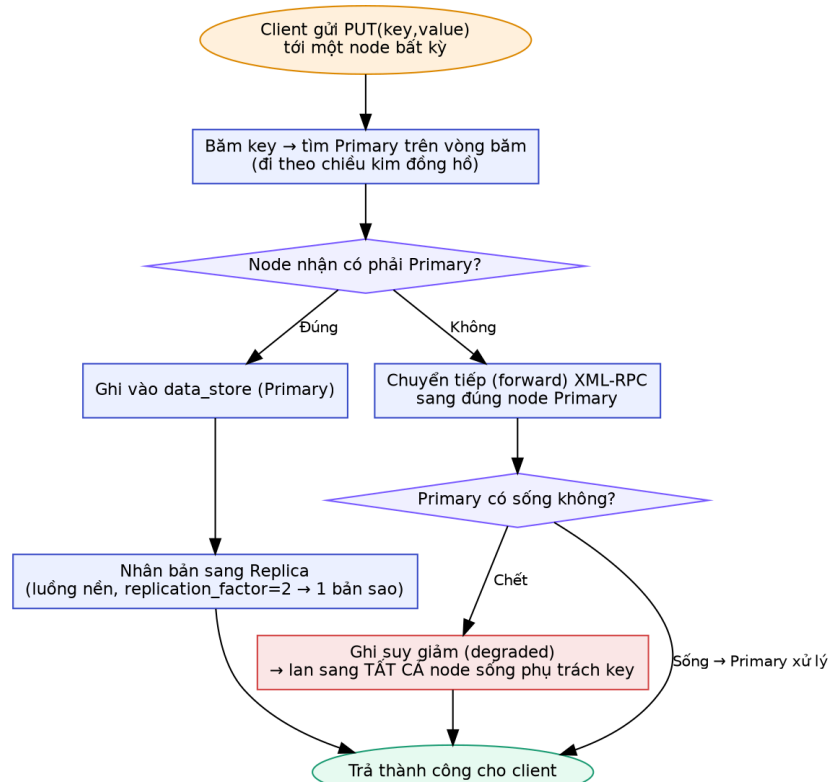
Primary trả lời client ngay sau khi ghi vào RAM cục bộ, không bắt buộc chờ nhân bản hoàn tất; việc nhân bản chạy bất đồng bộ. Trong vài phần mười giây dữ liệu Primary–Replica có thể chưa khớp tuyệt đối nhưng sẽ hội tụ. Lựa chọn này cho độ trễ ghi thấp và tăng tính sẵn sàng (Availability) theo định lý CAP.

4. Định tuyến và chuyển tiếp yêu cầu

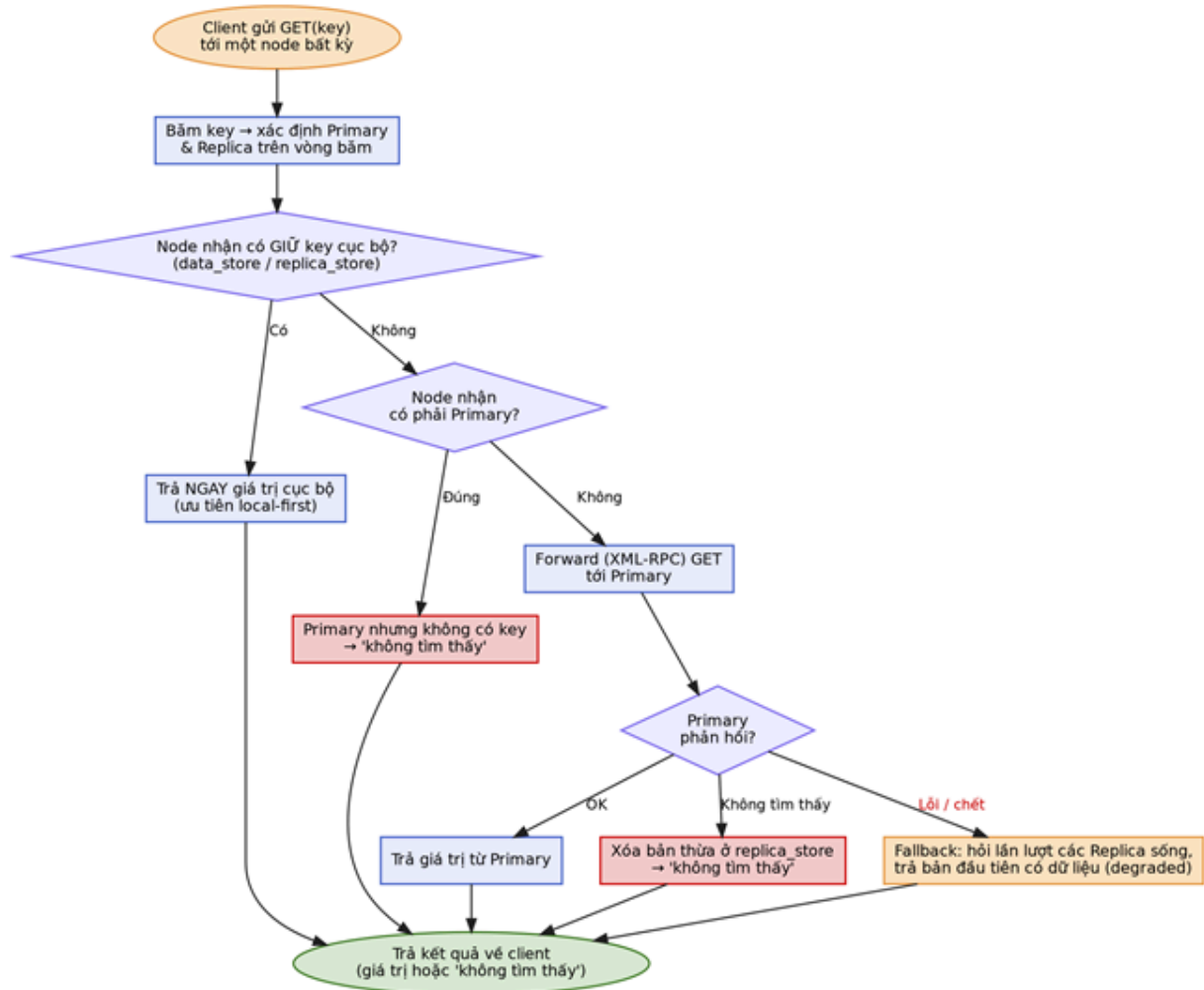
4.1. Định tuyến PUT và GET

Client có thể kết nối tới bất kỳ node nào, nên mỗi node tự định tuyến và chuyển tiếp. Với PUT: node tiếp nhận tính Primary của key; nếu chính nó là Primary thì ghi và nhân bản, ngược lại chuyển tiếp (forward) XML-RPC tới đúng Primary.

Với GET: node tiếp nhận trước hết kiểm tra bộ nhớ cục bộ — nếu đang giữ key (ở data_store hoặc replica_store) thì trả ngay giá trị đó (ưu tiên đọc cục bộ — local-first — để giảm độ trễ và số lần gọi mạng). Chỉ khi không có bản cục bộ và bản thân không phải Primary, node mới forward yêu cầu XML-RPC tới Primary; nếu Primary báo không có key thì xóa bản thừa và trả “không tìm thấy”, còn nếu Primary không phản hồi (lỗi/chết) thì fallback hỏi lần lượt các Replica sống để vẫn trả được dữ liệu (chế độ degraded).



Hình 4.1. Luồng xử lý lệnh PUT (gồm chuyển tiếp và ghi suy giảm khi Primary chết)



Hình 4.2. Luồng xử lý lệnh GET (ưu tiên đọc cục bộ local-first; nếu không có và không phải Primary thì forward tới Primary; fallback hỏi Replica sống khi Primary chết)

4.2. Dự phòng lỗi khi định tuyến (Failover Routing)

Khi một node trong cụm chết, hệ thống vẫn định tuyến không gây lỗi cho client: yêu cầu đọc/ghi tới key của node chết được phục vụ từ bản sao trên node sống (chế độ degraded). Quan trọng, khi Primary chết, một lệnh ghi mới được lan đồng thời tới TẤT CẢ node sống phụ trách key — tránh việc một bản sống giữ giá trị cũ rồi sau này ghi đè ngược lên giá trị mới.

5. Phát hiện lỗi và xử lý node chết

5.1. Heartbeat và phát hiện lỗi tự động

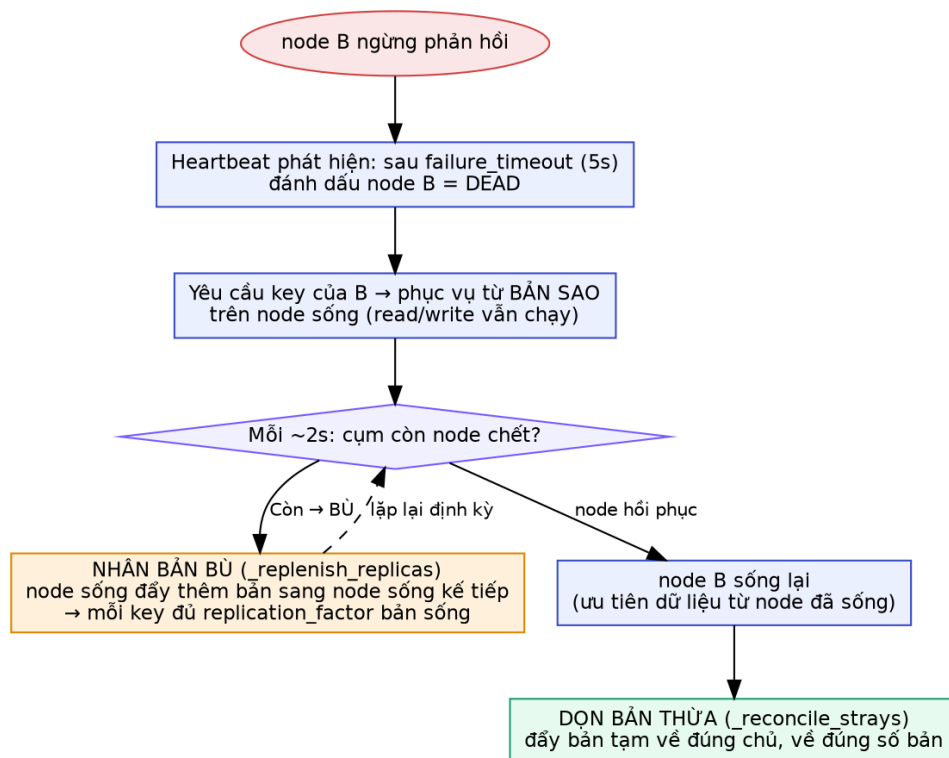
Mỗi node định kỳ gửi nhịp tim (mỗi 2 giây) tới các node lân cận. Một vòng lặp giám sát kiểm tra: nếu quá failure_timeout (5 giây) không nhận được nhịp tim từ một node đang ALIVE thì chuyển trạng thái node đó sang DEAD và kích hoạt các cơ chế bù dữ liệu.

5.2. Kịch bản khi node chết và cơ chế nhân bản bù

Khi node B chết: (1) Heartbeat phát hiện và đánh dấu DEAD; (2) các yêu cầu liên quan tới key của B được phục vụ tiếp từ bản sao trên node sống, client không thấy gián đoạn; (3) vì mỗi key của B tạm chỉ còn ít bản sống hơn yêu cầu, hệ thống CHỦ ĐỘNG nhân bản bù.

Nhân bản bù chạy ĐỊNH KỲ (mỗi ~2 giây), không phụ thuộc việc bất động khoảnh khắc node chết: mỗi node rà soát các key nó giữ, nếu một node sống phụ trách key còn thiếu bản thì đẩy thêm bản sang, đến khi mọi key đủ replication_factor bản sống. Nhờ vậy hệ thống luôn tự hồi tự về trạng thái đủ dự phòng.

Lưu ý kỹ thuật: bản sao mới được nhân từ bản còn sống trên các node khác, KHÔNG phải kéo từ node đã chết (vì node chết không kết nối được). Đây cũng là cách các hệ thống thực tế (Cassandra, Dynamo, HDFS) duy trì số bản sao.



Hình 5.1. Luồng failover và nhân bản bù khi một node chết, dọn bản thừa khi hồi phục

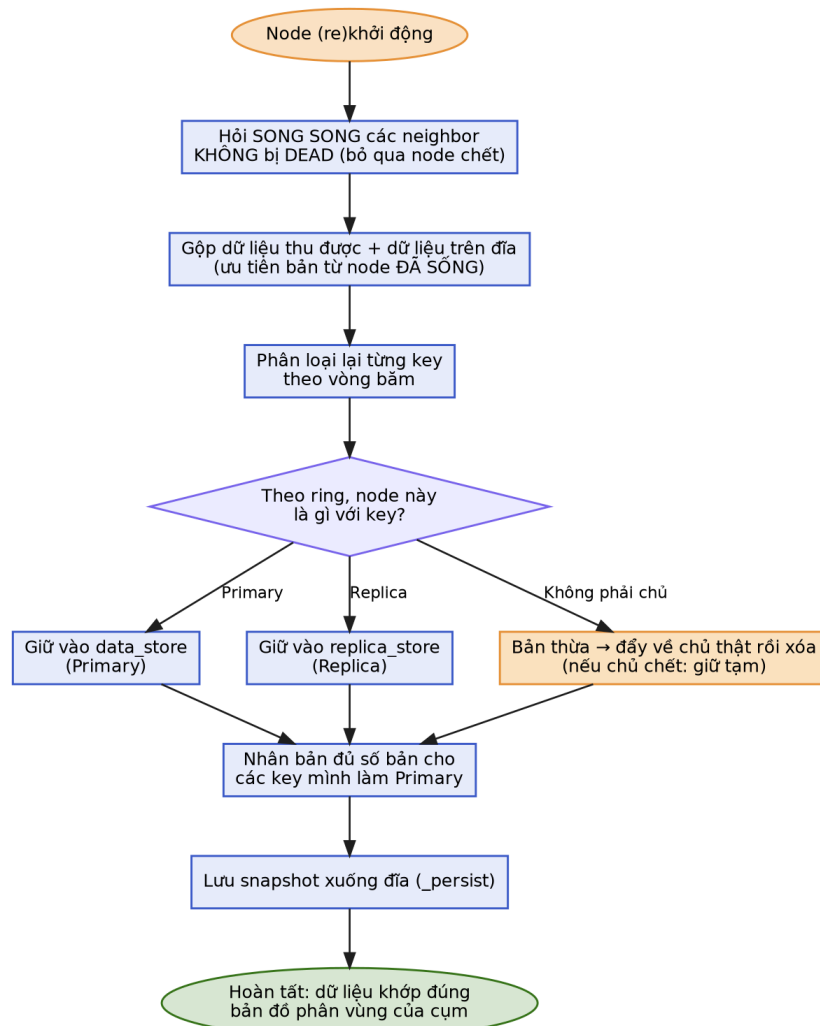
6. Khôi phục và đồng bộ lại dữ liệu

6.1. Đồng bộ khi khởi động và dọn bản thừa

Khi một node (re)khởi động, nó đồng bộ dữ liệu từ các node lân cận và phân loại

lại theo vòng băm: chỉ giữ những key mà nó là Primary hoặc Replica đúng theo ring. Các bản “thừa” (key mà nó không còn là chủ) được đẩy về đúng chủ rồi xóa, đảm bảo mỗi key đúng replication_factor bản — khắc phục triệt để lỗi nhân thừa bản sao từng xảy ra khi băm Đồng bộ.

Khi cụm khỏe mạnh trở lại (không còn node DEAD), cơ chế dọn bản thừa (_reconcile_strays) chạy định kỳ để gom các bản tạm sinh ra trong lúc sự cố về đúng số bản. Như vậy vòng đời khép kín: bình thường đủ 2 bản → node chết thì bù để vẫn đủ 2 bản sống → node hồi phục thì dọn về đúng 2 bản.



Hình 6.1. Luồng khôi phục và đồng bộ lại dữ liệu khi node (re) khởi động (phân loại theo vòng băm, dọn bản thừa, ưu tiên dữ liệu từ node đã sống)

6.2. Chống ghi đè dữ liệu cũ khi hồi phục

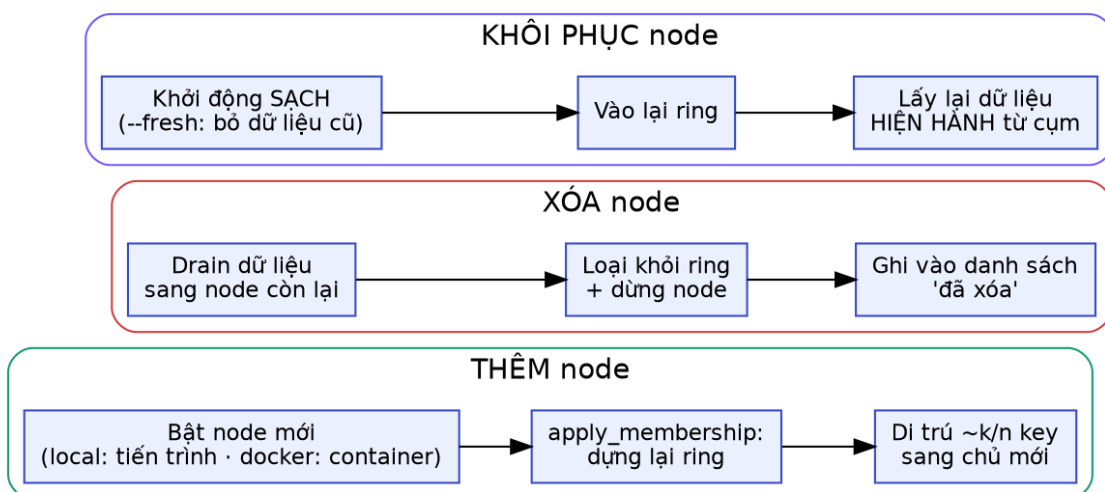
Trong lúc một node chết, mọi ghi mới chỉ tới được các node đang sống, nên dữ liệu trên đĩa của node vừa hồi phục có thể đã lỗi thời. Hệ thống áp dụng quy tắc: khi node hồi phục đồng bộ lại, ưu tiên giá trị từ các node ĐÃ SỐNG hơn dữ liệu

cũ trên đĩa của chính nó; dữ liệu cục bộ chỉ dùng để lấp chỗ trống. Nhờ vậy giá trị mới nhất luôn được giữ, không bị bản cũ ghi đè ngược.

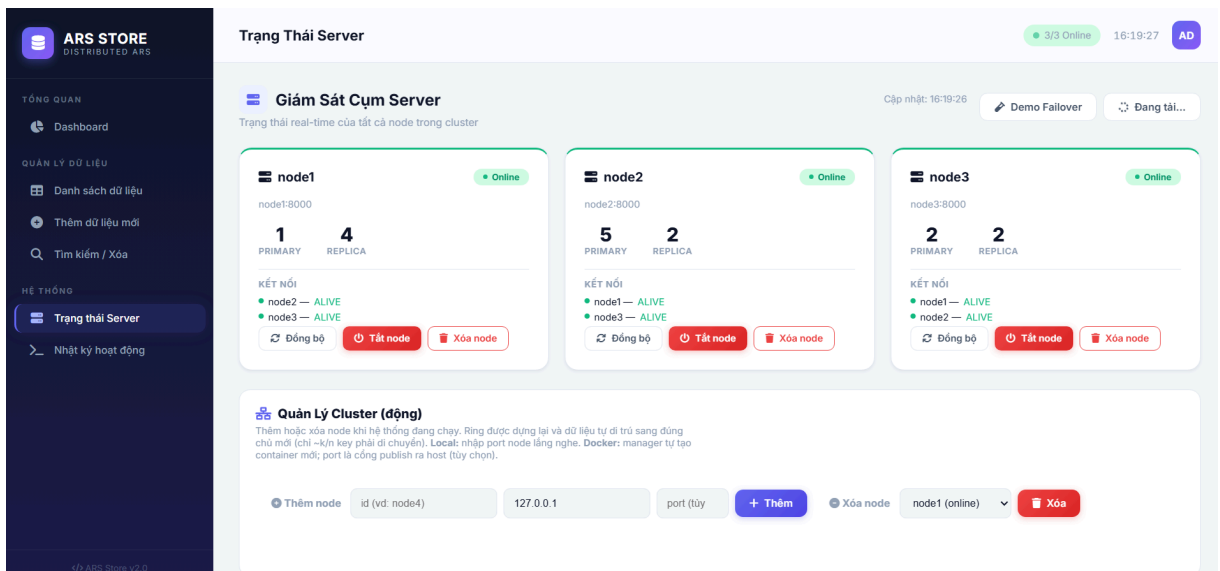
7. Quản lý cụm động: Thêm / Xóa / Khôi phục node

Hệ thống cho phép thay đổi thành phần cụm ngay khi đang chạy mà không mất dữ liệu. Mọi thao tác đều dừng lại vòng bấm và chỉ di trú phần dữ liệu cần thiết ($\approx k/n$ key) nhờ tính chất của consistent hashing.

- **THÊM node**: bật node mới (local: tiến trình; Docker: tạo container cùng image/mạng), dừng lại ring, chỉ một phần nhỏ key di trú sang chủ mới.
- **XÓA node**: drain (đẩy) toàn bộ dữ liệu của node sang các node còn lại, loại khỏi ring rồi dừng node; node sống trở thành chủ thật của các key đó.
- **KHÔI PHỤC node**: đưa node đã xóa (hoặc đang offline) trở lại với đúng ID, khởi động SẠCH (bỏ dữ liệu cũ có thể lỗi thời) và lấy lại dữ liệu HIỆN HÀNH từ cụm — phân biệt rõ với “Bật node” là tiếp tục với dữ liệu cũ trên đĩa.



Hình 7.1. Luồng Thêm / Xóa / Khôi phục node khi hệ thống đang chạy



Hình 7.2. Màn hình Trạng thái Server với bảng điều khiển Quản lý Cluster (động)

8. Phân tích theo định lý CAP

Trước sự đánh đổi giữa Nhất quán (Consistency), Sẵn sàng (Availability) và Chịu phân mảnh mạng (Partition Tolerance), hệ thống ưu tiên nhóm AP: luôn chấp nhận đọc/ghi (kể cả ở chế độ suy giảm khi có node chết) và đạt nhất quán cuối cùng (Eventual Consistency) thông qua nhân bản nền, nhân bản bù và dọn bản thừa. Đây là lựa chọn phù hợp cho một kho key-value đề cao tính sẵn sàng.

9. Ứng dụng quản trị trực quan (Web Dashboard)

Ứng dụng Flask cung cấp giao diện theo dõi thời gian thực và vận hành cụm: Dashboard tổng quan, Danh sách dữ liệu, Thêm dữ liệu, Tìm kiếm/Xóa, Trạng thái Server và Nhật ký hoạt động.

Màn hình Danh sách dữ liệu hiển thị mọi cặp key-value cùng Node gốc (Primary) và Bản sao (Replica); khi Primary đang offline, dòng dữ liệu được đánh dấu “đang phục vụ từ bản sao”:

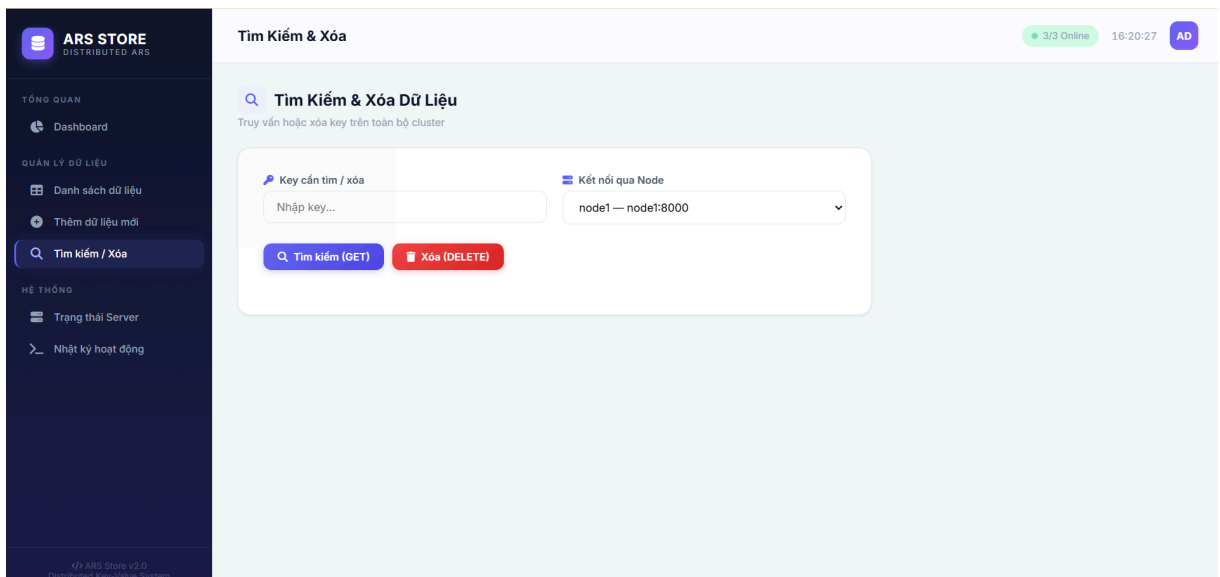
#	KEY	VALUE	NODE GỐC	BẢN SAO	THAO TÁC
1	data7	v7	node1	node3	Xóa
2	data1	v1	node2	node1	Xóa
3	data3	v3	node2	node1	Xóa
4	data4	v4	node2	node1	Xóa
5	data6	v6	node2	node1	Xóa
6	data2	v2	node2	node3	Xóa
7	data0	v0	node3	node2	Xóa
8	data5	v5	node3	node2	Xóa

Hình 9.1. Màn hình Danh sách dữ liệu (Primary / Replica của từng key)

Màn hình Thêm dữ liệu cho phép nhập key-value và chọn node tiếp nhận; hệ thống tự định tuyến tới đúng Primary:

Hình 9.2. Màn hình Thêm dữ liệu mới

Màn hình Tìm kiếm / Xóa cho phép truy vấn (GET) hoặc xóa (DELETE) một key trên toàn cụm:



Hình 9.3. Màn hình Tìm kiếm & Xóa dữ liệu

9.1. Tối ưu hiệu năng tải

Để giao diện phản hồi nhanh (mục tiêu 1–3 giây) ngay cả khi có node chết, các truy vấn trạng thái tới nhiều node được thực hiện SONG SONG với hạn thời gian (deadline) thay vì tuần tự; việc tra cứu định tuyến cho nhiều key được gộp vào một gọi RPC duy nhất; và các yêu cầu fetch phía trình duyệt được đặt timeout để nút “Đang tải” không bị kẹt. Kết quả đo: các API trạng thái phản hồi dưới ~15 mili-giây kể cả khi một node đang chết.

10. Cài đặt, triển khai và kết quả

10.1. Yêu cầu môi trường và cách cài đặt

Hệ thống chạy trên Python 3, dùng chủ yếu thư viện chuẩn (xmlrpc, threading) cho node và Flask cho ứng dụng Manager; toàn bộ cấu hình cụm khai báo tập trung trong cluster_config.json. Có hai cách triển khai. (1) Chạy cục bộ: khởi động từng node bằng lệnh “python node.py --node <NODE_ID> --config cluster_config.json --bind-host <IP>” (hoặc dùng script start_cluster), sau đó chạy Manager bằng “python manager_app.py”. (2) Chạy bằng Docker: dùng docker-compose (docker-compose.yml hoặc docker-compose.4node.yml) để dựng nhiều container node cùng image và cùng mạng; các script docker_local_up, docker_node_start hỗ trợ tạo/dùng container node ngay khi cụm đang chạy để thử nghiệm quản lý cụm động.

10.2. Kịch bản kiểm thử

Nhóm dùng bộ kịch bản kiểm thử (test_cluster.py, test_rpc.py, test_failover.py) và các script kiểm tra sức khỏe (healthcheck_node.py, healthcheck_manager.py)

để xác nhận: PUT/GET/DELETE định tuyến đúng Primary/Replica kể cả khi client gửi tới node không phải chủ; failover khi tắt một node bất kỳ; nhân bản bù khôi phục đủ số bản sau sự cố; thêm/xóa/khôi phục node khi đang chạy; và đồng bộ lại dữ liệu đúng theo vòng băm khi một node khởi động lại.

10.3. Kết quả đạt được

Cụm 3 node chạy ổn định trong môi trường mô phỏng; ba thao tác PUT/GET/DELETE hoạt động đúng kể cả khi yêu cầu được chuyển tiếp chéo giữa các node. Khi tắt một node, client vẫn đọc/ghi được nhờ phục vụ từ bản sao; sau vài giây, cơ chế nhân bản bù khôi phục đủ replication_factor bản sống; khi node hồi phục, dữ liệu được phân loại lại theo vòng băm và dọn bản thừa nên không dư bản. Giao diện Web Dashboard phản ánh trạng thái thời gian thực: sức khỏe cụm, danh sách key kèm Primary/Replica (đánh dấu khi đang phục vụ từ bản sao) và trực quan hóa vòng băm cùng tỉ lệ keyspace mỗi node. Sau khi tối ưu (gọi song song có deadline, gộp tra cứu định tuyến nhiều key vào một RPC), các API trạng thái phản hồi dưới ~15 mili-giây ngay cả khi một node đang chết. Các màn hình minh chứng kết quả được trình bày ở mục 9 (Hình 9.1–9.3) và vòng băm ở Hình 3.1–3.2.

11. Đánh giá hạn chế và đề xuất cải tiến

- Đồng bộ hóa: quy tắc “ưu tiên node đã sống” xử lý tốt trường hợp ghi-lúc-node-chết, nhưng chưa có version vector / nhãn thời gian cho từng key; trường hợp hiếm (node ghi xong rồi sập ngay trước khi kịp nhân bản) vẫn có thể mất bản ghi đó. Đề xuất: bổ sung Vector Clocks hoặc timestamp last-write-wins.
- Lưu trữ bền vững: dữ liệu chủ yếu trong RAM (có ghi snapshot JSON ra đĩa). Đề xuất: tăng cường Write-Ahead Logging (WAL) hoặc SQLite để phục hồi chính xác sau sự cố nguồn.
- Thuật toán đồng thuận: có thể tích hợp Raft/Paxos để đạt nhất quán mạnh khi cần.

12. Tính năng mới phát triển thêm

Ngoài phần lõi của một kho key-value phân tán (consistent hashing với virtual node, nhân bản, heartbeat/failover), nhóm phát triển thêm 02 tính năng mới gắn trực tiếp với khả năng xử lý phân tán dưới đây.

12.1. Tự phục hồi dữ liệu: nhân bản bù tự động và dọn bản thừa (self-healing)

Khi một node chết, các key do node đó phụ trách tạm thời thiếu bản sao. Một vòng lặp nền định kỳ (~2 giây) rà soát các key đang giữ và CHỦ ĐỘNG nhân bản bù (`_replenish_replicas`) sang các node sống kế tiếp trên vòng bấm cho tới khi mọi key đủ `replication_factor` bản sống; bản mới luôn được nhân từ các bản CÒN SỐNG chứ không kéo từ node đã chết. Khi cụm khỏe trở lại, cơ chế dọn bản thừa (`_reconcile_strays`) gom các bản tạm sinh ra trong lúc sự cố về đúng chủ theo ring để không dư bản. Nhờ vậy hệ thống tự hội tụ về trạng thái đủ dự phòng mà không cần can thiệp thủ công (chi tiết kỹ thuật ở mục 5.2 và 6.1). Ý nghĩa phân tán: nâng độ bền dữ liệu (durability) và duy trì khả năng chịu lỗi liên tục, tương tự cách Cassandra, Dynamo hay HDFS giữ đủ số bản sao.

12.2. Quản lý cụm động (dynamic membership): thêm / xóa / khôi phục node khi đang chạy

Manager có thể thay đổi thành phần cụm ngay khi hệ thống đang phục vụ: lời gọi `apply_membership` dựng lại vòng bấm theo membership mới và chỉ di trú phần dữ liệu cần thiết ($\approx k/n$ key) nhờ tính chất của consistent hashing. THÊM node (kể cả tạo container Docker mới rồi đưa vào ring); XÓA node (drain — đẩy dữ liệu sang các node còn lại — rồi loại khỏi ring và dừng node); KHÔI PHỤC node (khởi động sạch, bỏ dữ liệu cũ có thể lỗi thời và lấy lại dữ liệu hiện hành từ cụm). Mọi thao tác diễn ra không gián đoạn đọc/ghi và không mất dữ liệu (chi tiết ở mục 7). Ý nghĩa phân tán: cho phép co giãn quy mô trực tuyến (online elasticity) và bảo trì cuốn chiếu mà không phải dừng dịch vụ.

[Link github của nhóm](#)

Kết luận

Hệ thống lưu trữ Key-Value phân tán đã đạt đầy đủ yêu cầu đề ra: cụm nhiều node ngang hàng không phụ thuộc node trung tâm; Consistent Hashing với virtual node phân hoạch dữ liệu cân bằng; nhân bản đảm bảo chịu lỗi; Heartbeat phát hiện sự cố và tự động phục vụ từ bản sao; nhân bản bù liên tục và dọn bản thừa giúp luôn đủ số bản an toàn; quản lý cụm động (thêm/xóa/khôi phục node) khi đang chạy; cùng giao diện Web giám sát hiện đại. Hệ thống chạy ổn định trong môi trường mô phỏng và là nền tảng tốt để mở rộng với các thuật toán đồng thuận như Raft hoặc Paxos.